Such code is also, frankly, a pain to understand and maintain. Arbitrarily flicking between data fetching, application logic, and markup generation adds cognitive load to what is already a pretty difficult task. Nobody wants to deal with a pile of unstructured spaghetti code. Just as it's good OOP design to ensure that classes don't do too much (c.f. "blob" or "god object" anti-pattern `https://en.wikipedia.org/wiki/God_object`), and we have design patterns that give us good class hierarchies for certain problem spaces, we also have architectural patterns describing over-arching system design to help us avoid creating unmaintainable messes.

A Solution

A tried-and-tested architectural pattern for web applications is Model-View-Controller – MVC. This separates out three fundamental web application design aspects – processing the HTTP request itself (in a controller), fetching data and applying logic (the model), and rendering the resulting model back to the client (the view). Separating out the chief concerns of an application means that, amongst other benefits, different teams can work concurrently on the components without impacting each other's work. Other positive outcomes of MVC are increased testability (components can be tested in isolation) and better organization and maintainability of source code.

A good way to enforce that the view components of your application only contain logic concerned with final rendering output is to use a templating library. Although it's a bad idea to pepper your views with reams of code, you will still benefit from some programming tools as you form the output: iterating over arrays to write out `li` elements, varying layout depending on the presence of flags… or whatever it might be. Clearly your application will benefit from making use of at least *some* of PHP's features, and this is where a templating library comes in. This section will be using the popular Twig library (which is the default rendering component of the Symfony framework). Others are available, such as Blade (as used in the Laravel framework), but a lot of the concepts you'll learn here with Twig will transfer to others.

The Philosophy of Twig (and MVC)

Twig is a *Domain Specific Language* (DSL), which means that its features and syntax are restricted to a narrow problem space: templating. DSLs are implemented in a general-purpose language: in our case this is PHP (which started life as a limited set of CGI